

OpenCV * Python

Date: _____

Page: _____

① How to use open cv
`import cv2`

② Read the image.

`image = cv2.imread("filename.jpg", flag)`

`flag = 1` → for color (default)

`flag = 0` → for gray (black & white)

`flag = -1` → unchange

③ Display image.

`cv2.imshow("Image Title", image)` # open the window

`cv2.waitKey(0)` # wait for key for close

`cv2.destroyAllWindows()` # Close the window

④ Save image.

`cv2.imwrite("output image.png", image)`

⑤ image dimension

`image.shape` # height, width, channel

⑥ gray scale conversion.

`gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)`

⑦ resizing & scaling image (`cv2.resize()`)

`resized = cv2.resize(src, dsize, fx, fy, interpolation)`

`resized = cv2.resize(image, (300, 300))`

(width x height)

always saved resized image

⑧ Cropping Images Using Slicing in OpenCV

`Cropped_image = image[startY:endY, startX:endX]`

`Cropped_image = image[100:200, 50:150]`

⑨ Image Rotation and Flipping

`M = cv2.getRotationMatrix2D(center, angle, scale)`

`rotatedImage = cv2.warpAffine(image, M, (width, height))`

center = w/2, h/2 integer division

angle = 90, 180 # scale = 1.0, 2.0

`flipped_image = cv2.flip(image, flipCode)`

⑩ # flipCode 0 → vertical top → bottom

flipCode 1 → horizontally left → right

flipCode -1 → both.

⑩ Drawing a line - cv2.line()

`cv2.line(image, pt1, pt2, color, thickness)`

pt1 → (10, 50), pt2 → (50, 50)

color = (255, 0, 0), thickness = 4

⑪ Drawing a Rectangle - cv2.rectangle()

`cv2.rectangle(image, pt1, pt2, color, thickness)`

⑫ Drawing a circle - cv2.circle()

`cv2.circle(image, center, radius, color, thickness)`

thickness = -1 for fill the circle

⑬ Adding Text - cv2.putText()

`cv2.putText(image, text, org, font, font scale, color, thickness)`

org = (50, 300) → start from here

font = cv2.FONT_HERSHEY_SIMPLEX

font scale = (1.2) → size of the font

(14) Video Capture function - cv2.VideoCapture()

cap = cv2.VideoCapture(source)

Capture / laptop → source = 0

external → source = 1

Code:- import cv2

cap = cv2.VideoCapture(0)

while True:

ret, frame = cap.read() # ret = return (True/False)

if not ret:

print("could not read frame")

break

cv2.imshow("webcam feed", frame)

if cv2.waitKey(1) & 0xFF == ord('q'):

print("quitting...")

break

cap.release()

cv2.destroyAllWindows()

(15) Video recode function

cv2.VideoWriter(filename, codec, fps, frame size)

Code import cv2

camera = cv2.VideoCapture(0)

frame_width = int(camera.get(cv2.CAP_PROP_FRAME_WIDTH))

frame_height = int(camera.get(cv2.CAP_PROP_FRAME_HEIGHT))

codec = cv2.VideoWriter_fourcc(*'XVID')

recorder = cv2.VideoWriter("my_video.avi", codec, 20, (frame_width, frame_height))

while True:

success, image = camera.read()

if not success:


```

        break
    recorder.write(image)
    cv2.imshow("Recording live", image)
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break
    camera.release()
    recorder.release()
    cv2.destroyAllWindows()

```

⑩ Gaussian Blur

```

blurred_image = cv2.GaussianBlur(image, (kernel_size_x, kernel_size_y), sigma)

```

```

blurred = cv2.GaussianBlur(image, (7,7), 0)

```

kernel size (3,3) → light blur
(9,9) → strong blur
(21,21) → super blur

⑪ Median Blur

```

blurred = cv2.medianBlur(image, kernel_size)

```

⑫ ~~Sharpening~~ Sharpening

```

cv2.filter_2D(src, ddepth, kernel)

```

Code

```

import cv2

```

```

import numpy as np

```

```

image = cv2.imread('')

```

```

image = cv2.imread('Filtering & Blurring Image.png')

```

```

sharpen_kernel = np.array([

```

```

    [0, -1, 0],

```

```

    [-1, 5, -1],

```

```

    [0, -1, 0]

```

```

])

```

```

sharpened = cv2.filter_2D(image, -1, sharpen_kernel)

```


Contours: edge point of the images.

Date: _____

Page: _____

```
cv2.imshow("Original Image", image)
cv2.imshow("Sharpened Image", sharpened)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

(19) Canny Edge detection.
edges = cv2.Canny(image, threshold1, threshold2)

(20) Thresholding in Open CV
ThresholdImage = cv2.threshold(image, thresh-value, max-value, method)

(21) Bitwise operator in image.
bitwise-and = cv2.bitwise_and(img1, img2)
bitwise-or = cv2.bitwise_or(img1, img2)
bitwise-not = cv2.bitwise_not(img1)

Contours and Shape detection.

```
cv2.drawContours(image, contours, contour-index, color, thickness)
Code → import cv2
image = cv2.imread("tangle.png")
image = cv2.imread("tangle.png")
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
_, thresh = cv2.threshold(gray, 240, 255, cv2.THRESH-
    BINARY)
contours, hierarchy = cv2.findContours(thresh, cv2.RETR-
    TREE, cv2.CHAIN_APPROX_SIMPLE)
cv2.drawContours(img, contours, -1, (0, 255, 0), 3)
cv2.imshow("Contours", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```